

Cross-Schema Composition for Attestation-Native Chains

Typed Attestation References with Slashing-Aware Proof Propagation

Ligate Labs Research, Working Paper v0.2

Date: 2026-05-22

Contact: hello@ligate.io

Abstract

Ethereum smart contracts can reference arbitrary chain state by hash. The reference is well-typed if and only if the consumer contract enforces the type, which it does as application logic. There is no chain-level guarantee that a Solidity contract is consuming the right kind of input. Slashing propagation through references is also entirely application-level: contract A may invalidate state that contract B reads, and contract B has no native machinery to learn this. The result, across the EVM ecosystem, is a steady stream of integration bugs and a high cost of audit for any contract that composes with another contract’s state.

This paper specifies **chain-enforced typing** for attestation references and **slashing-aware proof propagation** through the schema dependency graph as a runtime primitive on Ligate Chain. A schema declares its input dependencies as part of its registration: which schemas it may reference, at which version constraints, with which predicate over the input payloads. The runtime rejects, at mempool admission, any attestation that references an input of the wrong type, the wrong version, or invalid status. When a referenced attestation transitions to REVOKED, SLASHED, or DEPENDENT-INVALID, the runtime propagates the transition through the dependency graph according to each schema’s declared cascade rule (strict or lazy), and the §5.5 termination theorem guarantees the cascade completes in bounded time.

Three contributions. First, we specify the protocol-level mechanism (§3 and §4): the schema-as-typed-graph model, the input-type-set with semver-constrained edges, the deterministic type predicate, and the runtime type check at mempool admission. Second, we prove the **slashing-cascade termination theorem (§5.5)**: under acyclic dependency graphs with depth d , the cascade triggered by one slash event terminates in $O(d)$ deterministic steps with bounded per-step compute, with explicit per-edge deduplication that prevents re-cascading through the same edge twice. Third, we analyze three attack families (type confusion, slash amplification, cycle-induced DoS) and bound the damage each can inflict (§8).

The mechanism is positioned as a **v2 protocol feature**. v1 of Ligate Chain ships with single-schema attestations only; cross-schema composition lands when 2-3 design partners have asked for it with concrete use-case descriptions matching the §6.2 template. This paper exists so the design space is captured before then, not as a roadmap commitment to ship. §6 is explicit about the use-case-validation gate: without validated design-partner demand, the mechanism remains specification-only.

1. Introduction

1.1 The Composition Thesis

[**v0.2:** Why typed cross-schema references are the right primitive for an attestation-native chain. The argument: attestations are claims about other things; many natural claims are claims about other claims (e.g., “this AI output was generated from this prompt by this model” is a claim that references a prompt attestation). Without chain-enforced typing, every consumer re-implements the type check, which means errors compound. With typing, the chain is the single source of truth for “is this reference valid.”]

1.2 Why Now (or Why Not)

[v0.2: Honest framing. v1 ships without this primitive. Single-schema attestations are sufficient for Themisra (proof of prompt), Mneme (wallet receipts), Iris (agent attestations), Kleidon (SaaS events). v2 territory; this paper exists to lock in the design before the engineering cycle, not to advocate for shipping it on day 1.]

1.3 The Type-Confusion Problem

[v0.2: Worked example. A consumer schema “proof-of-attribution” expects to reference a “proof-of-prompt” attestation. Without chain-enforced typing, the consumer takes any 32-byte hash that points to chain state. Adversary submits a hash pointing to a different schema (a Mneme transfer receipt, say). Consumer accepts; downstream apps treat the bogus attribution as canonical. With chain-enforced typing, the runtime rejects at attestation-time.]

1.4 The Central Question

[v0.2: What is the minimum typing and slashing-propagation primitive that makes cross-schema composition safe to use in production, without imposing v1 engineering cost on workloads that do not need it?]

1.5 Approach in Brief

[v0.2: Three-sentence preview. Schemas declare dependency edges in their registration. The runtime enforces type checks at attestation-time and propagates slashes through the dependency graph at slash-time. Cycle detection is static-by-default with an opt-in dynamic mode for advanced use cases.]

1.6 Contributions

[v0.2: Type system specification, slashing-cascade theorem, cycle-detection algorithm, security analysis under three attack families, formal comparison with Ethereum smart-contract references and EAS schema graphs.]

1.6.1 Status of Claims [v0.2: Same panel as PoUA v0.7 §1.6.1. The slashing-cascade termination theorem is a candidate for “proven”; the type-soundness claim is “bounded-under-stated-assumptions”; use-case fitness is “empirical-or-heuristic.”]

1.7 Scope and Non-Goals

[v0.2: In scope: typing and slashing propagation for cross-schema references on Ligate Chain. Out of scope: cross-chain references (separate paper), zero-knowledge predicate types (research-grade), fully-dependent type systems (would require a proof-search runtime; not appropriate for a chain).]

1.8 Document Structure

[v0.2: TOC walkthrough.]

2. Background and Related Work

2.1 Smart-Contract Reference Patterns

[v0.2: ERC-721 token references, ERC-1155 multi-token references, ERC-4907 rental references. All app-level; chain checks the hash but not the type.]

2.2 EAS Schema Graph

[v0.2: Ethereum Attestation Service has a similar graph structure, with schemas referencing schemas via UID. Type-check is application-level. Closest existing analog to what this paper specifies, but missing the chain-enforced typing.]

2.3 Capability-Secure Systems

[v0.2: Capability-based programming languages (E, Pony, Joe-E) provide compile-time enforcement of “object A can only invoke object B if it holds a capability.” Conceptual ancestor for “schema A can only reference schema B if dependency edge declared at registration.”]

2.4 Dependent Types in Programming Languages

[v0.2: Coq, Agda, Idris, Lean. Provide arbitrarily expressive types but require proof-search at type-check time. Inappropriate for a chain runtime; included for design-space context.]

2.5 Recursive Invalidation in Distributed Systems

[v0.2: Cache invalidation, cascading deletes in relational databases, transaction rollback in WAL systems. Each is a different point in the tradeoff space. The slashing-cascade design draws on cascading-delete semantics.]

3. System Model

This section formalises the typed-attestation-graph model: schemas as nodes, dependency edges as typed arrows, attestations as witnesses, and the validity state machine that supports slashing-aware cascades.

3.1 Schemas as Typed Graphs

A **schema** σ is a tuple

$$\sigma = (I_\sigma, O_\sigma, P_\sigma, \mathcal{A}_\sigma, V_\sigma)$$

where:

- I_σ is the **input-type set**: a finite list of (σ', v') pairs identifying schemas σ' at version constraints v' that this schema may reference (§4.2)
- O_σ is the **output payload type**: a structural schema (e.g., JSON Schema, Protobuf-like definition) for the attestation’s payload

- P_σ is the **type predicate**: a deterministic function $P_\sigma : I_\sigma^* \times O_\sigma \rightarrow \{\text{true}, \text{false}\}$ that evaluates input attestations and payload to a validity boolean (§4.3)
- $\mathcal{A}_\sigma$ is the **attestor set** required to sign attestations of this schema (inherited from the protocol’s existing schema primitive)
- V_σ is the **schema version**, a monotonic integer

A schema with $I_\sigma = \emptyset$ is a **leaf schema**: it does not reference other schemas. v0.7 PoUA chains support only leaf schemas; this paper specifies the extension to non-leaf schemas via the input-type-set I_σ .

3.2 Dependency Graph

The collection of registered schemas and their dependency edges forms a **directed graph**:

$$\mathcal{G} = (\Sigma, E)$$

where Σ is the set of registered schemas and $E \subseteq \Sigma \times \Sigma$ is the set of dependency edges. An edge $\sigma_a \rightarrow \sigma_b$ exists iff $(\sigma_b, v) \in I_{\sigma_a}$ for some version constraint v that admits σ_b ’s current version V_{σ_b} .

Acyclicity by default. Schema registration rejects edges that would close a cycle; cycle detection is computed at registration time via topological sort. This is the v0 default. §5.6 specifies an opt-in cyclic mode for advanced use cases.

Versioning expands the graph. When a schema upgrades from $V_{\sigma_b} = 1$ to $V_{\sigma_b} = 2$, the edges into σ_b are re-evaluated against the new version. Dependents that admit the new version (via subtyping, §4.5) keep their edges; dependents that require strict version match must be re-registered.

Graph properties. At v0 scale (a few hundred schemas), $|\Sigma|$ and $|E|$ are small; cycle-check and topological-sort costs are negligible. At v1+ scale (thousands of schemas), incremental cycle detection on each new registration becomes the cost driver; v0.2 of this paper specifies the algorithm.

3.3 Attestation as Witness

An **attestation** a of schema σ is the tuple

$$a = (\sigma, K^{\text{signer}}, \text{payload}_a, \text{refs}_a, t_a, s_a)$$

where:

- σ is the schema-id (which fixes $I_\sigma, O_\sigma, P_\sigma, \mathcal{A}_\sigma, V_\sigma$)
- K^{signer} is the signing key (typically the threshold-signature aggregation of $\mathcal{A}_\sigma$)
- payload_a is the payload conforming to O_σ
- refs_a is the **reference list**: $(\text{att-id}_1, \text{att-id}_2, \dots)$ indexing input attestations satisfying I_σ
- t_a is the inclusion height (block at which the attestation was finalized)
- s_a is the threshold signature

Witness semantics. An attestation a is a witness to “the predicate P_σ held over the referenced inputs and the stated payload at height t_a .” The chain stores the attestation; light clients can verify the predicate held by re-evaluating P_σ at any future point.

Empty references. Leaf schemas have $\text{refs}_a = ()$ (empty tuple). The runtime trivially passes the input-type check for leaf schemas; this is how PoUA v0.7’s existing schema primitive composes with this paper’s typed extension without requiring chain-state migration.

Reference-by-id, not by-content. refs_a uses canonical attestation-ids, not payload digests. The id binds to a specific on-chain attestation rather than any attestation with a matching payload, eliminating ambiguity when payloads collide across schemas.

3.4 Validity States

Attestations live in a **validity state machine** with four states:

```
VALID -> REVOKED           (revocation by signer or attester-set)
VALID -> SLASHED           (PoUA-side slashing event)
VALID -> DEPENDENT-INVALID (cascade from a non-VALID input)
```

State transitions.

- **VALID** is the initial state on inclusion. The runtime type check (§4.4) must pass for the attestation to enter VALID.
- **VALID** → **REVOKED** fires when the schema’s attester set issues a revocation transaction. Used for retraction (e.g., “this attestation was issued in error”).
- **VALID** → **SLASHED** fires when slashing rules per PoUA §4.5 detect misbehavior on the proposer or attester set. Slashing applies the reputation penalty and transitions any included attestations to SLASHED.
- **VALID** → **DEPENDENT-INVALID** fires when at least one of the attestation’s refs_a transitions out of VALID. The cascade rule (§5) determines whether the dependent attestation is auto-invalidated or whether application-layer logic decides.

Terminal states. REVOKED, SLASHED, and DEPENDENT-INVALID are terminal: no further transitions are possible. This simplifies the chain’s storage model (each attestation has a single state-bit set).

Cascade rules. Whether DEPENDENT-INVALID fires automatically at each ref-transition is governed by the schema’s declared cascade preference (§5.4). Strict cascade fires automatically; lazy cascade leaves the dependent VALID and exposes the ref-state via the read API for application-layer handling.

4. Type System

This section specifies how schemas declare types, how the runtime checks types, and how versioning interacts with the typed-graph model.

4.1 Schema Declaration

Schema registration submits a **schema-declaration object**:

```
SchemaDeclaration = {
  name:           string,    // canonical schema-id
  version:        int,       // monotonic version counter
  payload_schema: Type,      // structural schema
```

```

input_type_set:      List<(SchemaId, VersionConstraint)>,
predicate:           Bytecode, // deterministic boolean function
attestor_set_id:     Id,       // existing PoUA primitive
cascade_rule:        enum {STRICT, LAZY},
predicate_gas_limit: int,      // bounded compute cost (default 1000)
}

```

Field notes: **name** is the canonical schema-id (e.g. a `themisra.proof-of-prompt` schema at version `v1`); **payload_schema** is a structural type (JSON Schema or equivalent); **predicate** is a bytecode-encoded deterministic boolean function; **attestor_set_id** references the existing PoUA primitive.

Validation at registration time.

1. **name** is unique in Σ across all current versions.
2. **version** strictly exceeds all prior versions of **name**.
3. Each $(\sigma', v') \in \text{input_type_set}$ references an existing schema; cycle detection runs against $\mathcal{G}$ extended with the proposed edges (§3.2).
4. **predicate** is deterministic (verified by the runtime via dry-run on canonical inputs) and bounded by **predicate_gas_limit**.
5. **attestor_set_id** exists.
6. **cascade_rule** is one of the two enum values.

Failure of any check rejects the registration; partial registration is not allowed. Registration fees in \$AVOW apply per PoUA §3.

4.2 Input Type Set

The **input-type set** I_σ enumerates which schemas an attestation of σ may reference. Each entry is a (schema-id, version-constraint) pair:

$$I_\sigma = \{(\sigma_1, v_1), (\sigma_2, v_2), \dots, (\sigma_n, v_n)\}$$

where each v_i is one of:

- **Exact match** ($v_i = k$): only attestations of σ_i at version exactly k are admissible
- **Semver range** ($v_i = [k, k')$): versions $V \in [k, k')$ are admissible; supports forward-compatible consumers
- **Open upper** ($v_i = [k, \infty)$): all versions $\geq k$
- **Wildcard** ($v_i = *$): any registered version of σ_i . Discouraged; only use for cross-version-tolerant consumers (e.g., audit-log readers)

Why version constraints matter. A consumer schema declaring “I reference attestations of `themisra.proof-of-prompt/*`” accepts any future version of Themisra without re-registration. Convenient but risky: a v3 of Themisra with structurally-different payload could break the consumer’s predicate. Exact match prevents this at the cost of forcing re-registration on every input upgrade. Semver ranges balance the two.

Empty input-type set. $I_\sigma = \emptyset$ marks σ as a leaf schema (§3.1). Attestations of leaf schemas have $\text{refs}_a = ()$ and the type check trivially passes the input checks.

4.3 Type Predicate

The **type predicate** P_σ is a deterministic boolean function:

$$P_\sigma : I_\sigma^* \times \text{Payload}_\sigma \rightarrow \{\text{true}, \text{false}\}$$

evaluated at attestation-time. Inputs to the predicate:

- Each input attestation’s schema, version, and payload (read-only)
- The proposed attestation’s own payload

Predicate semantics:

- **Deterministic:** evaluating the same inputs in two different chain states must produce the same result. The predicate cannot read mutable chain state beyond the explicit input attestations.
- **Bounded compute:** the predicate’s gas cost is bounded by `predicate_gas_limit` (default 1000 gas-units; configurable per schema with governance bounds in `[100, 10000]`).
- **Total:** the predicate must terminate on all valid inputs. Non-termination is detected by the gas limit; predicates that exhaust gas are treated as returning `false` and the attestation is rejected at mempool admission.

Examples of useful predicates.

- “Each input attestation’s `model_id` field equals my own `model_id` field” (Themisra: prove the attribution chain consistently identifies the same model)
- “Each input attestation’s `timestamp` is monotonically older than mine” (audit-trail attestations: ensure causal ordering)
- “The sum of input `amount` fields equals my own `total` field” (financial attestations: conservation-of-value check)

Examples of disallowed predicates.

- “Look up the validator’s current reputation and gate on it” (reads mutable state)
- “Walk the attestation graph two hops deep” (unbounded recursion)
- “Hash a 1MB payload via SHA-256” (gas-bounded but potentially out-of-bound for `predicate_gas_limit`)

4.4 Runtime Type Check

When an attestation a of schema σ is submitted, the runtime performs the following checks at mempool admission (before block inclusion):

1. **Schema lookup.** σ exists in Σ at the registered version V_σ . Failure: reject “unknown schema.”
2. **Reference resolution.** For each ref $r_i \in \text{refs}_a$, look up the referenced attestation a_i and its schema σ_i , version V_{σ_i} . Failure: reject “dangling reference.”
3. **Input-type check.** For each (r_i, a_i, σ_i) , verify that $(\sigma_i, v) \in I_\sigma$ for some constraint v admitting V_{σ_i} . Failure: reject “input type mismatch.”
4. **Validity check.** For each a_i , verify that $a_i.\text{state} = \text{VALID}$. Failure: reject “stale reference” (the input is REVOKED, SLASHED, or DEPENDENT-INVALID).

5. **Predicate evaluation.** Evaluate $P_\sigma(a_1, \dots, a_n, \text{payload}_a)$. Failure: reject “predicate violated.”
6. **Existing PoUA checks.** Threshold signature verification per $\mathcal{A}_\sigma$, fee payment, etc.

All six must pass. Mempool rejection is preferable to block rejection: it avoids consuming block space and gives the submitter a clear failure signal.

Cost analysis. Steps 1-4 are $O(|\text{refs}_a|)$ state lookups; step 5 is bounded by `predicate_gas_limit`; step 6 is the existing PoUA cost. Per-attestation overhead vs leaf attestations is roughly 5-10% at typical reference counts ($|\text{refs}_a| \leq 5$).

Re-checking on cascade. When a referenced attestation transitions out of VALID, dependents already in VALID must be re-evaluated. The cascade rule (§5.4) decides automatic vs lazy re-evaluation. The check itself is identical to the admission-time check, run against the new state.

4.5 Versioning Semantics

When a schema upgrades from $V = k$ to $V = k + 1$, the chain must decide what happens to dependents that referenced $V = k$.

Strict mode. Dependents must re-reference: existing attestations that referenced $V = k$ are unaffected (still valid), but new attestations cannot reference the old version through this schema’s input-type set. This is the safest mode: the type contract is enforced at the exact-version level.

Subtyping mode. If $V = k + 1$ ’s payload schema is a structural superset of $V = k$ ’s (every field in $V = k$ exists with the same type in $V = k + 1$), and the predicate P_σ at $V = k + 1$ is a “weakening” of P_σ at $V = k$ (every input that satisfied the $V = k$ predicate also satisfies the $V = k + 1$ predicate), then $V = k + 1$ is a **subtype** of $V = k$. Existing dependents that referenced $V = k$ may continue to reference $V = k$ attestations, AND new attestations may reference $V = k + 1$ attestations under the same input-type-set entry.

Default: subtyping for backward-compatible upgrades; strict otherwise. A schema upgrade declares whether it is a subtype of its predecessor. The runtime verifies subtyping claims (structural superset checking is decidable; predicate weakening is detected by re-running P^k against the canonical inputs that satisfied P^{k+1} ; if any such input fails P^k , the upgrade is not a subtype).

Per-edge override. A consumer schema’s input-type set entry can override the default:

- (`themisra.proof-of-prompt/v1`, `exact`): strict, only $V = 1$
- (`themisra.proof-of-prompt/v1`, `subtype`): subtyping permitted (default for backward-compatible upgrades)

Migration. When a schema upgrades and existing dependents are not subtype-compatible, the runtime does NOT auto-migrate or auto-invalidate dependents. Existing attestations remain valid. New attestations from those dependents fail the input-type check (§4.4 step 3) until the dependent re-registers with updated input-type-set entries.

5. Slashing Propagation

The §3.4 validity state machine introduced DEPENDENT-INVALID as the cascade state. This section specifies the cascade mechanics: what fires when an attestation transitions out of VALID, how dependents respond, how the runtime keeps termination bounded under arbitrary graph shapes, and how concurrent slash events interleave.

5.1 The Cascade Question

When a referenced attestation a transitions to REVOKED, SLASHED, or DEPENDENT-INVALID, the chain has three candidate behaviors for a 's dependents:

1. **Strict cascade:** every dependent immediately transitions to DEPENDENT-INVALID at the slash root's transition block.
2. **Lazy cascade:** dependents stay in VALID; the read API exposes the input's invalid status; application logic decides what to do.
3. **Hybrid:** per-schema declaration at registration picks strict or lazy.

None of the three is universally correct. Strict is right when the dependent's claim *relies on* the input's truth and any reader of the dependent should see invalidation propagate (e.g., a financial attestation referencing a sovereign-identity proof). Lazy is right when the dependent's claim is *informationally* dependent but functionally autonomous (e.g., an audit-trail attestation that references a parent event but is itself a standalone receipt of "I observed this"). The hybrid is what we ship: schemas declare their cascade preference at registration (§4.1's `cascade_rule` field), and the runtime enforces it deterministically per-schema.

This section formalizes each candidate, specifies how the cascade algorithm runs, proves the termination theorem (§5.5), and bounds adversarial behavior under concurrent slashes (§5.7).

5.2 Strict Cascade

Under strict cascade, when an attestation a transitions to a non-VALID state at block t , every attestation b such that $a \in \text{refs}_b$ transitions to DEPENDENT-INVALID at block $t + 1$ (or at t if the runtime processes the cascade in the same block; see §5.7 for ordering semantics). The transition is automatic: no on-chain action by the dependent's submitter is required.

Semantics for readers. A read of b after the cascade returns `state = DEPENDENT_INVALID` with metadata `caused_by = a` and `caused_at = t`. Light clients can verify the cascade by re-evaluating: was a a referenced input of b at attestation time, and is a 's current state non-VALID? If both yes, b 's state is correctly DEPENDENT-INVALID. This is one extra state-tree lookup per cascade depth, $O(1)$ per verification.

Recovery. A dependent submitter cannot un-invalidate b . The recovery path is to submit a new attestation b' with corrected references (pointing to a still-VALID equivalent of a , or omitting the broken reference entirely). b remains DEPENDENT-INVALID forever; b' is a fresh attestation that, if valid, supersedes b for downstream consumers.

Cost. Per slash root, the chain pays for re-evaluating each dependent's state. With strict cascade, this is $O(|\text{descendants}|)$ state-tree writes per slash root, where descendants are the transitive set of attestations reachable through dependency edges from the slash root. §5.5 bounds this in terms of dependency depth.

Use cases. Schemas where the dependent’s claim is meaningless if any input is invalid:

- Financial conservation-of-value attestations (the total field depends on all input amounts; if any input is slashed, the total is unverified)
- Identity composition: “this DAO membership references this identity proof”; if the identity is revoked, the membership is no longer evidence-backed
- Multi-party signature: “all parties consented” via references; any party-slash invalidates the multi

5.3 Lazy Cascade

Under lazy cascade, when an input *a* transitions to a non-VALID state, dependents stay in VALID. The chain records the slash root’s transition but does not propagate. The read API returns `state = VALID`, with metadata listing each input’s current state. Application code decides what to do with the information.

Semantics for readers. A read of *b* returns `state = VALID` plus `inputs = [(a, state_a, block_a), ...]`. A reader who wants strict semantics must walk the input states themselves: if any input is non-VALID, treat *b* as application-layer-invalid. Readers who want lazy semantics ignore the input states and trust *b*’s standalone payload.

Recovery. Not required at the chain level. The dependent submitter may choose to re-submit with corrected references, but the chain does not force it. *b* remains VALID indefinitely even if every input is slashed.

Cost. Per slash root, the chain pays for the slash root’s own transition only. No cascade-induced writes. State-tree footprint is minimal.

Use cases. Schemas where the dependent’s claim is independently meaningful and the references are informational:

- Audit-trail attestations: “I observed event X” with X as input; if X is slashed, my observation is still a true record of what I observed
- Bulk AI-provenance attestations: a Themisra session-end attestation that references each turn’s individual attestations; if one turn is slashed, the session-end is still a valid record of the session
- Reputation summaries: “this validator did X, Y, Z” where X, Y, Z reference individual events; one event invalidation does not change the summary’s accuracy

5.4 Configurable Per-Schema

The schema-declaration object (§4.1) carries a `cascade_rule` field with two values: `STRICT` or `LAZY`. The choice is made by the schema author at registration and applies to all attestations of that schema.

Why per-schema and not per-attestation. Per-attestation cascade rules would require the runtime to read each attestation’s rule at cascade time, doubling state-lookup cost. Per-schema rules are read once at schema lookup; subsequent attestations of the schema inherit. The trade-off: less flexibility, more efficiency. A schema author who wants both behaviors registers two schemas (e.g., `themisra.attribution.strict/v1` and `themisra.attribution.lazy/v1`).

Default for new schemas. `STRICT`. This errs on the side of safety: a schema author who has

not thought carefully about cascade semantics gets the stronger correctness guarantee. To opt into LAZY, the author must explicitly declare it.

Governance bound. Cascade rule is immutable post-registration; changing it would invalidate the type contract for existing dependents. A schema that wants to switch must register a new schema (with a new schema-id) and let consumers migrate.

Interaction with §4.5 versioning. A schema upgrade may not change cascade rule; the upgrade is rejected if it tries. This is a stricter rule than payload subtyping: cascade rule is a structural property of the schema’s economic semantics and cannot be retrofitted.

5.5 Slashing-Cascade Termination Theorem

Claim (Theorem 1). Under any acyclic dependency graph $\mathcal{G}$, when a single attestation a transitions from VALID to non-VALID at block t , the strict-cascade algorithm terminates in $O(d)$ deterministic steps, where d is the maximum depth of any descendant of a in $\mathcal{G}$.

Setup. Define the **descendant set** $D(a) = \{b : a \rightarrow^* b \text{ in the dependency graph}\}$ (transitive closure of “references” through strict-cascade edges). The cascade algorithm processes $D(a)$ in BFS order from a :

```
Q := [a]
visited := {a}
while Q non-empty:
  x := Q.dequeue()
  for each y such that x in refs(y) and y.cascade_rule = STRICT:
    if y not in visited:
      transition y to DEPENDENT_INVALID
      visited.add(y)
      Q.enqueue(y)
return |visited| # total number of cascaded transitions
```

Proof.

Termination. Each iteration of the while loop removes one element from Q and may add some new elements. An element y is added to Q at most once, because the `if y not in visited` guard prevents re-enqueueing. The total number of enqueue operations is therefore $|D(a)|$, which is finite (the chain has finitely many attestations). The loop terminates.

Bounded steps. The BFS visits $D(a)$ in layer order. Each layer corresponds to a depth-level in $\mathcal{G}$ rooted at a . The maximum number of layers is d by definition. Within each layer, the work per element is $O(\text{outdegree})$ for edge enumeration plus $O(1)$ for the state transition. Summing over layers gives total work $O(|D(a)| \cdot \text{outdegree})$. For the BFS depth itself (the number of sequential dependency layers traversed), the bound is exactly d .

Determinism. The BFS order is deterministic given a canonical ordering of attestation ids (by hash). The cascade transitions for the same slash root, processed in the same block, produce identical chain states.

Acyclicity is necessary. Without acyclicity, the BFS could revisit an attestation through a different path. The `visited` set prevents this, but the termination guarantee would still depend on per-edge deduplication; §5.6 addresses the dynamic-cyclic case explicitly.

Corollary (depth bound). At v0, the chain enforces a maximum dependency depth $d_{\max} = 8$ at schema-registration time (cycle-detection in §4.4 already requires walking the graph; depth-bound is computed in the same walk). This bounds any cascade to $\leq |D(a)|$ transitions, with $|D(a)| \leq \text{outdegree}_{\max}^{d_{\max}}$. At typical scale (outdegree ≤ 3 , depth ≤ 8), $|D(a)| \leq 6561$ attestations per slash root.

Corollary (cost amortization). The cascade gas cost is charged to the slash root’s submitter (the party whose attestation was slashed), not to dependent submitters. This is the §8.3 economic defense against slash-amplification attacks: an adversary who can slash a heavily-referenced attestation faces the cost of cascading all its descendants, even though the descendants are unrelated to the adversary’s goal. The cost ratio scales with $|D(a)|$ and the per-attestation transition cost.

5.6 Cycle Handling

§5.5’s termination guarantee assumes the dependency graph is acyclic. Two cycle-handling modes are specified.

Static mode (v0 default). Schema registration rejects edges that would close a cycle. Cycle detection is computed at registration time via DFS over the existing graph $\mathcal{G}$ extended with the proposed new edges. The check is $O(|E|)$ time per registration, where $|E|$ is the number of existing edges. At v0 scale ($|E| \leq 1000$), this is microseconds.

A registrant who wants a cyclic dependency must use the dynamic mode (below) or restructure their schemas to break the cycle. The default rejection is the safer choice: cycles are rare in practice and introduce significant complexity in cascade semantics.

Dynamic mode (opt-in, v1+). Cycles are permitted at registration, but each schema must declare additional parameters:

- **cycle_break_rule:** enum {NO_REVISIT_EDGE, NO_REVISIT_NODE}. The runtime uses the rule to prevent infinite cascades.
- **max_cascade_depth:** integer in $[1, 100]$. Hard cap on cascade BFS depth, even if the graph allows deeper traversal.

Under NO_REVISIT_EDGE, the cascade BFS does not traverse the same edge twice; under NO_REVISIT_NODE, it does not visit the same attestation twice. Both prevent infinite loops; NO_REVISIT_NODE is stricter and is the recommended default for dynamic-mode schemas.

Why dynamic mode is deferred to v1+. Dynamic cycles complicate the §5.5 termination proof (the depth d becomes the **max_cascade_depth** parameter, not a graph property) and introduce edge cases in concurrent cascades (§5.7). Production use cases for cyclic schemas are rare; v0 ships without dynamic mode and waits for design-partner demand before enabling it.

5.7 Concurrent Invalidation Races

Two slash events can occur in the same block. Consider: attestation a slashed for misbehavior; attestation b revoked by the schema’s attestor set; both a and b are referenced by attestation c . What state does c end up in?

Deterministic ordering. Cascade events within a block are processed in canonical attestation-id order (by hash). The runtime sorts the slash roots and processes their cascades sequentially

within the block. The order is fully determined by chain state and the block contents; no validator-discretion is introduced.

Deduplication on dependents. When c is reached during the cascade for a , it transitions to `DEPENDENT-INVALID` with `caused_by = a`. When c is reached again during the cascade for b , the runtime detects c is already `DEPENDENT-INVALID` (terminal state per §3.4) and skips. The cascade for b continues with c ’s descendants but does not re-transition c .

Race semantics. The “`caused_by`” field reflects the first slash root that reached c in canonical order, not necessarily the “logical” root of the invalidation chain. This is a deliberate design choice: chains derive truth from the canonical block ordering, not from external semantics. Applications that need richer attribution can read the full input-state list (the `inputs` metadata from §5.3’s lazy-cascade read) to reconstruct the full invalidation pattern.

Gas accounting. Each slash root pays gas for its own cascade traversal. If a and b both reach c , a pays for the transition of c (it gets there first in canonical order); b pays only for its own subtree minus the intersection with a ’s subtree. This is the §8.3 amplification defense: the second slasher gets a partial discount, but does not get a free ride.

Cross-block races. If a and b are slashed in different blocks, the cascades are sequential by block order. The earlier block’s cascade completes (within its block) before the later block’s cascade begins. No interleaving between blocks; chain finality enforces the order.

6. Use Cases (Design-Partner-Validated)

6.1 The Use-Case-Validation Gate

[**v0.2:** This section is **the gate** for v0.2 authoring. Cross-schema composition is theoretically nice but practically expensive. v0.2 ships only when 2-3 design partners have asked for it specifically and submitted concrete use-case descriptions matching this section’s template.]

6.2 Use Case Template

[**v0.2:** For each use case: (a) what the consumer schema produces, (b) which input schema is required, (c) why the type contract must be chain-enforced (not application-enforced), (d) what slashing-cascade behavior is needed, (e) failure mode if the chain doesn’t enforce.]

6.3 Hypothetical Use Cases (Pre-Validation)

[**v0.2:** Three hypotheticals, marked clearly as not-yet-validated: 1. AI attribution: “this output was produced from this prompt” requires reference to prompt attestation; type-confusion attack would be a major safety issue 2. Multi-party signing: “all parties consented” requires references to individual party attestations; slash on any party invalidates the multi 3. Proof of audit: “this RWA reserve was audited” references the auditor’s qualified attestation; auditor-slash invalidates pending audit-claims]

6.4 What This Section Looks Like at v0.2

[**v0.2:** Three real use cases from real partners. Without them, this section stays placeholder and v0.2 does not ship. Authoring is gated.]

7. Comparison

7.1 vs Ethereum Smart-Contract References

[v0.2: EVM contracts reference state by hash; type-checking is application-level. Quantitative: gas cost per reference, type-soundness, slash propagation cost.]

7.2 vs EAS Schema Graph

[v0.2: EAS supports schema-references via UID. Type-checking is application-level (no chain enforcement). Slash propagation is application-level (no chain cascade).]

7.3 vs Capability-Secure Languages

[v0.2: E / Pony / Joe-E provide compile-time capability checks. Closer in spirit to this paper's typing but operates at language level, not chain level. Conceptual ancestor; not directly comparable.]

7.4 vs Cosmos IBC Light Clients

[v0.2: IBC supports cross-chain references with light-client verification. Different problem (cross-chain) but informs the on-chain encoding of “this attestation is provably from chain X at height H.”]

8. Security Analysis

8.1 Threat Models

[v0.2: Three attack families: 1. Type-confusion attacks: adversary references wrong-type attestation; expects acceptance from buggy consumer 2. Slash-amplification attacks: adversary triggers slash on a heavily-referenced attestation; expects cascade to disrupt unrelated workflows 3. Cycle-induced DoS: adversary registers cyclic schema graph; expects cascade to stall or loop]

8.2 Type-Confusion Defense

[v0.2: Chain-enforced typing rejects at attestation-time per §4.4. Defense is structural; no economic exposure.]

8.3 Slash-Amplification Defense

[v0.2: Slashing-cascade depth bounded; slash root pays the cascade cost (gas), not dependents. Defense is economic.]

8.4 Cycle-DoS Defense

[v0.2: Static-by-default cycle rejection at registration. Even in dynamic mode, bounded cascade depth + per-edge deduplication prevent infinite loops.]

9. Limitations and Future Work

9.1 Cross-Chain Composition

[v0.2: Out of scope. A separate paper covers attestation-graph references across IBC-connected chains.]

9.2 Predicate Expressiveness

[v0.2: Type predicates P_σ are bounded-compute boolean functions. More expressive predicates (zero-knowledge, recursive) are research-grade and deferred.]

9.3 Proof-Carrying Attestations

[v0.2: Attestations could carry SNARK proofs of input validity rather than referencing. Different design point with different tradeoffs; not a v0.2 deliverable.]

9.4 Privacy-Preserving References

[v0.2: A reference reveals the dependency edge in the public chain state. Use cases requiring private references (e.g., consumer schema does not want to disclose which input it consumed) need additional cryptography. Future work.]

10. Conclusion

[v0.2: Recap. Chain-enforced typing for attestation references, slashing-aware proof propagation through the schema dependency graph, and explicit non-shipment in v1. The mechanism is interesting and probably correct, but the engineering cost is justified only by validated design-partner demand. v2 territory.]

References

[v0.2: Ethereum Attestation Service docs, capability-secure languages survey, dependent types references (Coq / Agda / Idris / Lean), cascading-delete semantics in DB literature, plus standard PoUA references.]

Appendix A: Simulator Validation Plan

[v0.2: What prototypes/cross-schema-composition-sim/ will contain. Test harness for type-check correctness, slashing-cascade termination, cycle-detection, concurrent-invalidation race deduplication. Cross-language test vectors for the canonical type-check predicate encoding.]

Appendix B: Formal Definitions

[v0.2: Restated definitions of schema, dependency edge, attestation, type predicate, validity state, cascade rule, in formal notation.]